



智能招投标审查平台 - 技术设计文档

基于 LangGraph 多Agent架构的智能招投标审查系统

项目版本
V1.0

创建日期
2026-03-27

文档状态
初稿

技术架构
前后端分离 + LangGraph多Agent

目录

- 项目概述
- 技术架构
- 项目结构
- 技术栈清单
- Agent设计
- 四层记忆机制
- LLM可扩展架构
- 配置管理
- 功能模块
- 部署方案
- 项目文件清单
- 附录

1. 项目概述

1.1 项目背景

随着我国公共资源交易市场化改革不断深化，招投标领域面临着日益严峻的合规监管挑战，包括串标围标、虚假资质、关联公司围标等问题。传统人工审查方式效率低、覆盖面窄、标准不统一。

1.2 建设目标

通过AI Agent技术实现招投标文件的智能审查，提高工作效率，减少人工疏漏，为决策提供全面参考意见。

1.3 建设原则

- API-first**：核心服务通过API暴露，支持多种客户端和外部系统集成
- Agent架构**：基于LangGraph实现多Agent协作，支持功能灵活扩展
- 四层记忆机制**：每个Agent具有独立的四层记忆能力
- 安全可控**：确保系统和数据安全，支持私有化部署
- 循序渐进**：分阶段建设，逐步完善
- 人机协同**：系统输出仅作为辅助参考，不替代法定责任主体

2. 技术架构

2.1 整体架构图





2.2 架构特性

1. 前后端分离：Vue3前端通过REST API与后端通信
2. 多Agent协作：基于LangGraph实现复杂的Agent workflow
3. 四层记忆机制：每个Agent具有独立的记忆能力
4. LLM可扩展：支持前期阿里云API、后期私有化部署
5. 统一文档解析：基于Unstructured.io的文档处理
6. 多数据库协同：PostgreSQL + Qdrant + Neo4j + Redis

3. 项目结构

```
bid_tool_agents/
├── frontend/                                # Vue3 前端项目
│   ├── src/
│   │   ├── api/                            # API调用层
│   │   ├── components/                    # 公共组件
│   │   ├── pages/                         # 页面
│   │   ├── stores/                       # Pinia状态管理
│   │   ├── types/                        # TypeScript类型
│   │   └── utils/                        # 工具函数
│   └──
├── backend/                                # FastAPI + LangGraph 后端
│   ├── app/
│   │   ├── api/                          # API路由
│   │   ├── agents/                      # ★ LangGraph Agent
│   │   │   ├── base/                   # Agent基类 + 四层记忆
│   │   │   ├── coordinator/           # 协调器Agent
│   │   │   ├── parser/                # 文档解析Agent
│   │   │   ├── compliance/            # 合规审查Agent
│   │   │   ├── comparison/            # 比对分析Agent
│   │   │   ├── qualification/         # 资质核验Agent
│   │   │   ├── risk/                  # 风险识别Agent
│   │   │   ├── evaluation/            # 辅助评标Agent
│   │   │   ├── expert/                # 专家抽取Agent
│   │   │   ├── archive/               # 档案管理Agent
│   │   │   └── statistics/            # 统计分析Agent
│   │   ├── skills/                     # 共享Skills
│   │   ├── tools/                      # 底层Tools
│   │   │   ├── database/               # 数据库连接
│   │   │   ├── document/              # 文档处理
│   │   │   └── llm/                   # LLM可扩展架构
│   │   ├── memory/                     # 全局共享记忆层
│   │   ├── models/                    # 数据模型
│   │   └── services/                  # 业务服务
│   ├── config/
│   ├── pyproject.toml
│   └── Dockerfile
├──
├── knowledge_base/                        # 知识库文件
│   ├── laws_regulations/               # 法律法规
│   ├── internal_rules/                 # 内部规则
│   ├── policies/                       # 制度
│   ├── qualifications/                 # 资质证书
│   ├── cases/                          # 案例库
│   ├── experts/                        # 专家库
│   ├── templates/                     # 模板库
│   └── sensitive_words/                # 错敏词库
├──
├── storage/                             # 文件存储
└── tests/                               # 测试
```

4. 技术栈清单

4.1 前端技术栈

类别	技术	版本	用途
框架	Vue.js	3.4+	核心框架
语言	TypeScript	5.x	类型安全
构建工具	Vite	5.x	快速构建
路由	Vue Router	4.x	页面路由
状态管理	Pinia	2.x	状态管理
UI组件库	Naive UI	2.x	UI组件（TS原生）
图表库	ApexCharts	-	数据可视化
HTTP客户端	Axios	1.x	API调用

4.2 后端技术栈

类别	技术	版本	用途
API框架	FastAPI	0.109+	REST API
Agent框架	LangGraph	0.1+	多Agent编排
数据验证	Pydantic	2.x	数据模型
数据库ORM	SQLAlchemy	2.x	数据库操作
异步任务	Celery	5.x	异步任务队列
日志	Loguru	-	日志记录

4.3 AI/LLM技术栈

类别	技术	版本	用途
LLM框架	LangChain	0.1+	LLM调用封装
Agent编排	LangGraph	0.1+	Agent workflow
Embedding	BGE	1.x	中文向量化
Token计算	tiktoken	-	Token计数

4.4 文档解析技术栈

类别	技术	用途
统一解析	Unstructured.io	PDF/Word/HTML统一解析
PDF处理	marker-pdf	PDF转Markdown
OCR识别	PaddleOCR	扫描件文字识别
Word解析	python-docx	Word文件解析
分词	jieba	中文分词

4.5 数据存储技术栈

类别	技术	用途
关系数据库	PostgreSQL	业务数据、Agent记忆
向量数据库	Qdrant	语义检索、相似度匹配
图数据库	Neo4j	企业关联、风险传播
缓存	Redis	工作记忆、会话缓存
对象存储	MinIO	文件存储

5. Agent设计

5.1 Agent列表

Agent	职责	核心技能
Coordinator	意图识别、任务分解、调度 汇总	intent_recognition, task_decomposition
Parser	PDF解析、OCR识别、文本 提取	pdf_parse, ocr_recognize
Compliance	合规性检查、问题识别	law_retrieval, compliance_check
Comparison	文本相似度、响应性比对	similarity_analysis, graph_query
Qualification	资质核验、有效期检查	info_extraction, validity_check
Risk	围串标检测、风险识别	risk_detection, similarity_analysis
Evaluation	评分标准、客观项分析	scoring_config, objective_analysis
Expert	专家库管理、随机抽取	expert_retrieval, random_selection
Archive	任务跟踪、整改记录、归档	task_management, archive_management
Statistics	统计分析、风险看板	statistics_analysis, risk_summary

5.2 Agent基类

```
from abc import ABC, abstractmethod
from typing import Any, Dict, Optional
from langgraph.graph import StateGraph
from .memory import AgentMemory

class BaseAgent(ABC):
    """Agent抽象基类，所有Agent继承此类"""

    def __init__(self, agent_id: str, name: str):
        self.agent_id = agent_id
        self.name = name
        self.memory = AgentMemory(agent_id)
        self.graph = self._build_graph()

    @abstractmethod
    def _build_graph(self) -> StateGraph:
        """构建LangGraph"""
        pass
```

```

@abstractmethod
async def process(self, input_data: Any) -> Dict:
    """处理输入"""
    pass

def get_memory(self) -> AgentMemory:
    return self.memory

```

5.3 LangGraph示例 (Coordinator)

```

from langgraph.graph import StateGraph, END
from langgraph.checkpoint.postgres import PostgresSaver
from .nodes import (
    intent_recognition_node,
    task_decomposition_node,
    agent_dispatch_node,
    result_aggregation_node
)
from .state import CoordinatorState

def create_coordinator_graph(checkpoint_db_url: str):
    """创建协调器Agent的LangGraph"""

    checkpointer = PostgresSaver.from_conn_string(checkpoint_db_url)
    workflow = StateGraph(CoordinatorState)

    # 添加节点
    workflow.add_node("intent_recognition", intent_recognition_node)
    workflow.add_node("task_decomposition", task_decomposition_node)
    workflow.add_node("agent_dispatch", agent_dispatch_node)
    workflow.add_node("result_aggregation", result_aggregation_node)

    # 添加边
    workflow.add_edge("intent_recognition", "task_decomposition")
    workflow.add_edge("task_decomposition", "agent_dispatch")
    workflow.add_edge("agent_dispatch", "result_aggregation")
    workflow.add_edge("result_aggregation", END)

    # 编译
    return workflow.compile(
        checkpointer=checkpointer,
        interrupt_before=["agent_dispatch"]  # 支持人工确认
    )

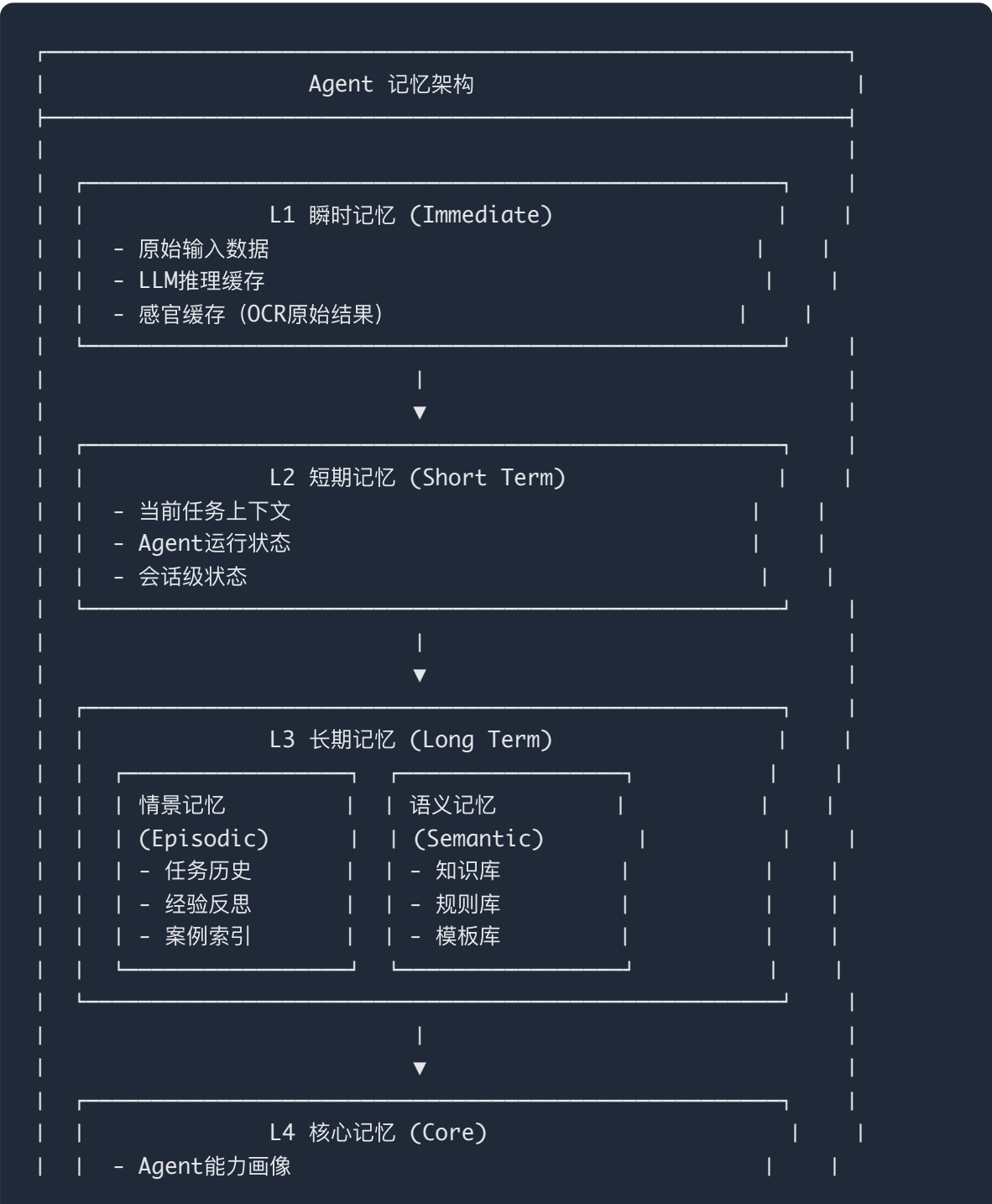
```

6. 四层记忆机制

6.1 记忆层级定义

层级	名称	生命周期	淘汰策略	存储介质
L1	瞬时记忆	< 1分钟	TTL自动过期	内存
L2	短期记忆	任务周期	任务结束清	Redis
L3	长期记忆	月~年	按重要性保留	PostgreSQL + Qdrant
L4	核心记忆	永久	只增不减，人工触发更新	PostgreSQL

6.2 记忆结构



```
| | - 记忆精华（关键经验） | |
| | - 决策模式 | |
| | - Agent间共享上下文 | |
| |_____
|
|_____
```

7. LLM可扩展架构

7.1 LLM类型定义

```
from enum import Enum
from dataclasses import dataclass
from typing import Optional

class LLMProvider(Enum):
    """LLM提供者类型"""
    ALI_QWEN = "ali_qwen" # 阿里云API
    PRIVATE_QWEN = "private_qwen" # 千问私有化
    PRIVATE_DEEPSEEK = "private_deepseek"

class LLMModel(Enum):
    """具体模型枚举"""
    QWEN_MAX = "qwen-max"
    QWEN_PLUS = "qwen-plus"
    QWEN_TURBO = "qwen-turbo"
    QWEN3_14B = "qwen3-14b"
    QWEN3_72B = "qwen3-72b"
    DEEPSEEK_V3 = "deepseek-v3"

@dataclass
class LLMConfig:
    """LLM配置"""
    provider: LLMProvider
    model: LLMModel
    api_base: Optional[str] = None
    api_key: Optional[str] = None
    max_tokens: int = 4096
    temperature: float = 0.7
    timeout: int = 120
    retry_times: int = 3
```

7.2 LLM管理器

```

from typing import Optional, Dict, List
from .base import BaseLLM
from .types import LLMConfig, LLMPProvider
from .registry import LLMRegistry

class LLMManager:
    """LLM管理器 - 统一管理LLM实例，支持多实例、热切换、负载均衡"""

    def __init__(self):
        self._instances: Dict[str, BaseLLM] = {}
        self._default: Optional[str] = None

    def add_llm(self, name: str, config: LLMConfig) -> None:
        instance = LLMRegistry.get(config)
        self._instances[name] = instance
        if self._default is None:
            self._default = name

    def set_default(self, name: str) -> None:
        self._default = name

    def get_llm(self, name: Optional[str] = None) -> BaseLLM:
        if name is None:
            name = self._default
        return self._instances[name]

    async def chat(self, messages: List[Dict], **kwargs) -> str:
        return await self.get_llm().chat(messages, **kwargs)

    async def embeddings(self, texts: List[str]) -> List[List[float]]:
        return await self.get_llm().embeddings(texts)

```

8. 配置管理

8.1 配置结构

```

config/
├─ settings.py          # 配置类定义
├─ development.yaml     # 开发环境
├─ production.yaml      # 生产环境
└─ testing.yaml         # 测试环境

```

8.2 配置项

配置项	开发环境	测试环境	生产环境
数据库	localhost/dev	localhost/test	云数据库
Redis	localhost	localhost/db1	云Redis
向量库	localhost	localhost	云Qdrant
图数据库	localhost	localhost	云Neo4j
LLM	阿里云API	阿里云Turbo	私有化千问
日志	DEBUG	DEBUG	INFO

9. 功能模块

9.1 一期功能（P0）

模块	功能
文件智能解析	文件上传、OCR、结构化解析、表格提取、原文定位
招标文件检测	错敏词、合规性、条款完整性、评标办法、一致性检查
投标文件审查	完整性、缺漏、响应性、格式、日期/金额/数量检查
招投标响应比对	要求提取、响应提取、映射、未响应/弱响应/偏离项识别
资质有效性核验	企业/人员/证书提取、有效期核验、单位一致性
围串标风险识别	相似度分析、雷同段落识别、联系方式/地址重合、报价离散度
知识库管理	法规库、规则库、错敏词库、模板库、规则生效管理

9.2 二期功能（P1）

模块	功能
辅助评标	评分标准配置、客观项分析、评分建议
专家随机抽取	专家库管理、随机抽取、回避过滤

模块	功能
档案管理	任务跟踪、整改记录、报告归档
统计分析	风险总览、问题统计、整改分析、报表导出

10. 部署方案

10.1 Docker Compose架构

```
# docker-compose.yml
version: '3.8'

services:
  # 前端
  frontend:
    build: ./frontend
    ports:
      - "3000:3000"

  # 后端API
  backend:
    build: ./backend
    ports:
      - "8000:8000"
    environment:
      - ENV=production
    depends_on:
      - postgres
      - redis
      - qdrant
      - neo4j

  # PostgreSQL
  postgres:
    image: postgres:15
    ports:
      - "5432:5432"

  # Redis
  redis:
    image: redis:7
    ports:
      - "6379:6379"

  # Qdrant
```

```
qdrant:
  image: qdrant/qdrant
  ports:
    - "6333:6333"

# Neo4j
neo4j:
  image: neo4j:5
  ports:
    - "7474:7474"
    - "7687:7687"
```

10.2 启动方式

```
# 开发环境
cd frontend && npm install && npm run dev
cd backend && pip install -r requirements.txt && uvicorn app.main:app --r

# 生产环境
docker-compose up -d
```

11. 项目文件清单

目录/文件	说明
frontend/	Vue3前端项目
backend/	FastAPI后端项目
backend/app/agents/	LangGraph Agent核心
backend/app/agents/base/memory/	四层记忆基类
backend/app/agents/*/memory/	各Agent专属记忆
backend/app/skills/	共享Skills
backend/app/tools/llm/	LLM可扩展架构
backend/app/tools/document/	文档解析工具

目录/文件	说明
backend/app/memory/	全局共享记忆层
config/	配置文件
knowledge_base/	知识库文件
storage/	文件存储
tests/	测试

12. 附录

12.1 术语定义

术语	定义
LangGraph	基于LangChain的多Agent编排框架
Agent	智能体，能够自主完成特定任务的AI程序
Skill	可被多个Agent复用的能力模块
Tool	底层工具调用
LLM	大语言模型
OCR	光学字符识别

12.2 参考资料

- [LangGraph官方文档](#)
- [Unstructured.io官方文档](#)
- [PaddleOCR官方文档](#)
- [Qdrant官方文档](#)
- [Neo4j官方文档](#)

